

```
import pandas as pd
import numpy as np
```

```
RAW_PATH = "social_media_screentime_mental_health_2026.csv"
df = pd.read_csv(RAW_PATH)
```

```
print("=" * 70)
print("STEP 1: INITIAL DATA PROFILE")
print("=" * 70)
print(f"Shape: {df.shape[0]:,} rows x {df.shape[1]} columns\n")

print("Data types:")
print(df.dtypes)

print("\nMissing values per column:")
missing = df.isna().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_report = pd.DataFrame({"missing_count": missing, "missing_pct": missing_pct})
print(missing_report[missing_report["missing_count"] > 0])

print(f"\nFully duplicated rows: {df.duplicated().sum()}")
print(f"Duplicate participant_id values: {df['participant_id'].duplicated().sum()}")
```

[Show hidden output](#)

```
print("\n" + "=" * 70)
print("STEP 2: OUTLIER DETECTION (IQR method)")
print("=" * 70)

def flag_outliers_iqr(series: pd.Series, k: float = 1.5) -> np.ndarray:
    """
    Returns a boolean numpy array flagging values outside [Q1 - k*IQR, Q3 + k*IQR].
    Vectorized – no row-by-row looping.
    """
    q1, q3 = np.nanpercentile(series, [25, 75])
    iqr = q3 - q1
    lower_bound = q1 - k * iqr
    upper_bound = q3 + k * iqr
    return (series < lower_bound) | (series > upper_bound)

numeric_cols_to_check = [
    "age", "daily_screen_hours", "daily_notifications", "avg_sleep_hours",
    "anxiety_score_0to27", "low_mood_score_0to27", "minutes_to_first_check_after_waking"
]

outlier_summary = {}
for col in numeric_cols_to_check:
    flags = flag_outliers_iqr(df[col])
    outlier_summary[col] = int(np.sum(flags))

print("Outlier counts (IQR method, k=1.5):")
for col, count in outlier_summary.items():
    print(f" {col:40s}: {count} flagged ({count/len(df)*100:.2f}%)")

df["is_screen_hours_outlier"] = flag_outliers_iqr(df["daily_screen_hours"])
```

[Show hidden output](#)

```
print("\n" + "=" * 70)
print("STEP 3: HANDLING MISSING VALUES")
print("=" * 70)

before_gender_na = df["gender"].isna().sum()
df["gender"] = df["gender"].fillna("Not specified")
print(f"gender: filled {before_gender_na} missing values with 'Not specified'")

before_sleep_na = df["avg_sleep_hours"].isna().sum()
df["sleep_hours_imputed"] = df["avg_sleep_hours"].isna() # audit flag

group_medians = df.groupby("wellbeing_band")["avg_sleep_hours"].transform("median")
df["avg_sleep_hours"] = np.where(
    df["avg_sleep_hours"].isna(),
    group_medians,
    df["avg_sleep_hours"]
)
print(f"avg_sleep_hours: imputed {before_sleep_na} missing values "
      f"using group median (by wellbeing_band)")
```

```
remaining_na = df.isna().sum().sum()
print(f"\nTotal remaining missing values after cleaning: {remaining_na}")
```

[Show hidden output](#)

```
print("\n" + "=" * 70)
print("STEP 4: FEATURE ENGINEERING")
print("=" * 70)

age_conditions = [
    df["age"] < 18,
    df["age"] <= 24,
    df["age"] <= 34,
    df["age"] <= 44,
    df["age"] <= 54,
]
age_labels = ["Under 18", "18-24", "25-34", "35-44", "45-54"]
df["age_group"] = np.select(age_conditions, age_labels, default="55+")

screen_conditions = [
    df["daily_screen_hours"] < 2,
    df["daily_screen_hours"] < 4,
    df["daily_screen_hours"] < 6,
]
screen_labels = ["Low (<2h)", "Moderate (2-4h)", "High (4-6h)"]
df["screen_time_tier"] = np.select(screen_conditions, screen_labels, default="Very High (6h+)")

check_conditions = [
    df["minutes_to_first_check_after_waking"] <= 5,
    df["minutes_to_first_check_after_waking"] <= 15,
    df["minutes_to_first_check_after_waking"] <= 30,
]
check_labels = ["Within 5 min", "6-15 min", "16-30 min"]
df["first_check_bucket"] = np.select(check_conditions, check_labels, default="30+ min")

df["distress_index"] = np.round(
    (
        (df["anxiety_score_0to27"] / 27 * 10)
        + (df["low_mood_score_0to27"] / 27 * 10)
        + (10 - df["life_satisfaction_1to10"])
    ) / 3,
    2
)

df["at_risk_flag"] = np.where(df["wellbeing_band"] == "At-risk", 1, 0)

df["digital_detox_attempted"] = np.where(
    df["attempted_digital_detox"].astype(str).str.startswith("Yes"), "Yes", "No"
)

df["avg_sleep_hours"] = df["avg_sleep_hours"].round(2)

print("New columns added:")
new_cols = ["age_group", "screen_time_tier", "first_check_bucket",
            "distress_index", "at_risk_flag", "digital_detox_attempted",
            "sleep_hours_imputed", "is_screen_hours_outlier"]
for c in new_cols:
    print(f" - {c}")
```

[Show hidden output](#)

```
print("\n" + "=" * 70)
print("STEP 5: POST-CLEANING VALIDATION")
print("=" * 70)

assert df.isna().sum().sum() == 0, "Missing values still present!"
assert df["participant_id"].is_unique, "Duplicate participant IDs found!"
assert df["age_group"].isna().sum() == 0, "Age group assignment failed for some rows!"
assert (df["distress_index"] >= 0).all() and (df["distress_index"] <= 10).all(), \
    "Distress index out of expected 0-10 range!"

print("All validation checks passed:")
print(" - No missing values")
print(" - No duplicate participant IDs")
print(" - All engineered categorical fields fully populated")
print(" - Distress index within expected 0-10 bounds")

cols_with_missing = (missing_report["missing_count"] > 0).sum()
print(f"\nFinal shape: {df.shape[0]:,} rows x {df.shape[1]} columns "
      f"({started_with {cols_with_missing} columns with missing data, now 0})")
```

[Show hidden output](#)

```
OUTPUT_PATH = "social_media_screentime_mental_health_2026_CLEANED.csv"
df.to_csv(OUTPUT_PATH, index=False)
print(f"\nCleaned file saved to: {OUTPUT_PATH}")
```

Cleaned file saved to: social_media_screentime_mental_health_2026_CLEANED.csv